

AI-Powered Leave Application System — Architecture & Design

Prototype design document Channel: Microsoft Teams · Parsing: LLM (natural language) · Date: 2026-06-07

1. Goal

Replace the manual, multi-field web form (login → open *Apply / Schedule Leave* → pick dates → select absence type → add comments → upload document → submit) with a **single natural-language message in Microsoft Teams**.

Today

Employee logs into PeopleSoft → navigates to *Apply / Schedule Leave* → fills Start Date, Absence Name, Comments, attachment → submits.

Proposed

Employee types in Teams: *"I'm down with fever, need sick leave tomorrow and the day after, doctor's note attached."* The bot understands it, fills the form behind the scenes, shows a confirmation card, and submits on approval.

2. What the form actually needs (mapped from presentUI.JPG)

The AI's only job is to produce a valid, complete version of this record. Every NL message must be resolved into these fields:

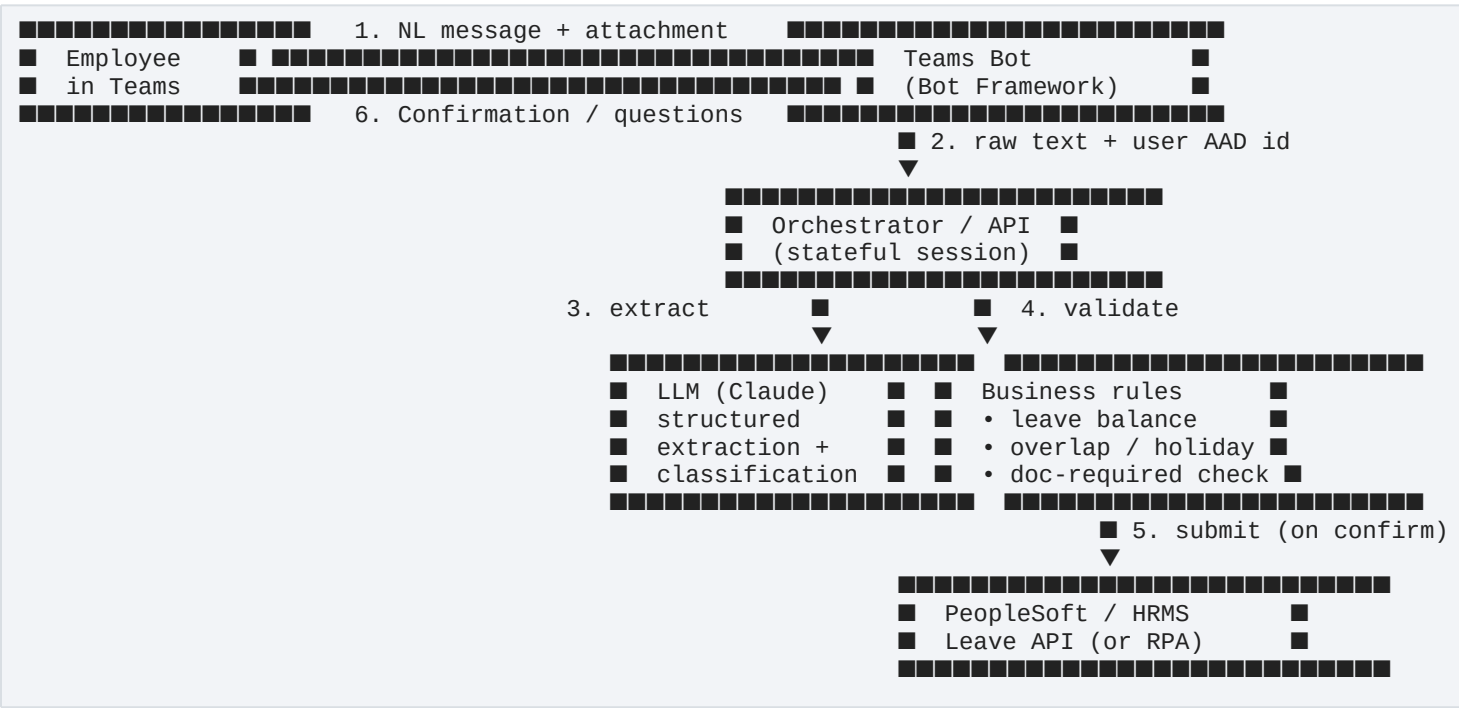
Form field	Required	Type / source	How AI fills it
Start Date	Yes	Date	Parse absolute/relative dates ("tomorrow", "next Mon", "10th") → ISO date in employee's timezone
End Date / Duration	Yes*	Date or day count (implied by the system)	Derive from "2 days", "Mon–Tue", "till Friday"; default 1 day if a single date is given
Absence Name	Yes	Dropdown — fixed enum of leave codes	Classify intent → map to a valid code (Casual / Sick / Privilege-Earned / Comp-off / WFH / LOP). Never invent a value.

Requestor Comments	No	Free text	Summarise the reason from the message
Document Upload	No	File attachment	Pull any file the user attached in Teams; flag if a doc is <i>expected</i> (e.g. sick leave > N days) but missing
Employee identity	Auto	Logged-in user	Derived from the Teams Azure AD identity — never typed by the user

* The screenshot shows only Start Date, but absence records always carry a duration/end. The design treats duration as required and confirms it with the user.

Key design rule: the LLM **extracts and classifies**, it does **not** decide policy. Balance checks, eligibility, and approval routing stay in deterministic backend code.

3. End-to-end flow



Step by step

- Capture** — Employee sends a chat message (and optional file) to the leave bot in Teams.
- Identify** — Bot reads the sender's Azure AD identity → maps to employee ID. No login form.
- Extract** — Orchestrator sends the message + conversation history to the LLM, which returns a **structured JSON** leave request (see §5).
- Validate** — Deterministic code checks: valid leave code, balance available, no overlap with existing/holiday/weekend, attachment present when policy requires one.

5. **Confirm** — Bot replies with an **Adaptive Card** showing the parsed request and Confirm / Edit buttons. Nothing is submitted until the user confirms.
6. **Submit** — On confirm, the request is pushed to the HRMS leave API (or RPA fallback). Bot returns the request ID and status; manager gets the usual approval notification.

4. Component architecture

Layer	Component	Responsibility	Tech (suggested)
Channel	Teams bot	Receive messages/files, render Adaptive Cards, handle buttons	Azure Bot Service + Bot Framework SDK (Python/Node), registered as a Teams app
Identity	AAD auth	Resolve Teams user → employee ID; SSO	Azure AD / Entra ID, OAuth on-behalf-of
Orchestrator	Conversation/session service	Hold multi-turn state, call LLM, call rules, drive confirmation	Python (FastAPI) or Node service
AI	LLM extraction	NL → structured leave JSON, ask clarifying questions	Claude API (claude-opus-4-8 / claude-sonnet-4-6) with tool/structured output
Rules	Policy engine	Balance, overlap, holidays, doc-required, leave-code validity	Deterministic service + HRMS lookups
Integration	HRMS connector	Submit absence, read balances/history	PeopleSoft Component Interface / REST, or RPA on the existing UI
Store	Session + audit	Pending drafts, conversation logs, audit trail	Redis (session) + relational DB (audit)

Why a separate Orchestrator? Teams, the LLM, and PeopleSoft each change independently. Keeping the business logic in the middle means you can swap Teams for Gmail later, or PeopleSoft for another HRMS, without touching the AI layer.

5. The structured contract (LLM output schema)

The LLM is forced to return **only** this shape (via tool-use / structured output). This is the bridge between fuzzy language and the rigid form.

```
{
  "intent": "apply_leave | check_balance | view_history | cancel_leave | other",
  "leave_request": {
    "start_date": "2026-06-08",           // ISO; resolved from relative phrases
    "end_date": "2026-06-09",           // ISO; = start_date if single day
    "duration_days": 2,                  // derived; supports 0.5 for half-day
    "half_day": false,
    "absence_code": "SICK",              // MUST be one of the allowed enum (see §6)
    "comments": "Fever, doctor's note attached",
    "has_attachment": true
  },
  "confidence": 0.0,                    // model self-rating; low → ask, don't assume
  "missing_or_ambiguous": ["end_date"], // fields the bot must clarify
  "clarifying_question": "Is this for one day or do you need both Monday and Tuesday off?"
}
```

Design principles: - **Closed enum for absence_code**. The model picks from a list the backend supplies; an unknown reason maps to `null` + a clarifying question, never a guessed code. - **Confidence + missing_or_ambiguous** drive the conversation. The orchestrator only proceeds to the confirmation card when nothing is missing; otherwise it asks the model's `clarifying_question`. - **Dates resolved server-side context**. The prompt is given *today's date* and the employee's timezone so "next Friday" is deterministic.

6. AI / prompt design

System prompt (essence):

You are a leave-application assistant. Convert the employee's message into a structured leave request using only the provided `absence_code` values and the given current date/timezone. Extract dates, duration, leave type, and reason. If anything required is missing or ambiguous, do **not** guess — set it to `null`, list it in `missing_or_ambiguous`, and propose one short clarifying question. You never approve, reject, or comment on leave policy.

Dynamic context injected each call: - `current_date`, `employee_timezone` - Allowed `absence_code` list with descriptions, e.g. SICK (illness), CASUAL (short personal), EARNED/PRIVILEGE (planned vacation), COMP_OFF (worked holiday), WFH, LOP (loss of pay) — *sourced live from HRMS so it never drifts*. - Recent conversation turns (for multi-turn clarification).

Why an LLM and not regex: real messages are messy — *"out for my sister's wedding the last week of this month"*, *"wfh tomorrow, kid's sick"*, *"take Friday off and Monday too"*. The model handles relative dates, ranges, half-days, and implicit leave-type inference; regex cannot do this reliably.

Guardrails - Closed enums + server-side validation of every field the model returns. - Confidence threshold → clarify instead of assume. - The model **cannot** submit; it only fills a draft. Submission is a separate, user-confirmed action. - All extractions logged for audit and prompt-tuning.

7. Conversation patterns

Happy path

User: Need 2 days sick leave from tomorrow, attaching the medical certificate. [file]
 Bot: Here's what I'll submit:

- Type: Sick Leave
 - Dates: 08-Jun-2026 → 09-Jun-2026 (2 days)
 - Comment: "Fever, medical certificate attached"
 - Attachment: medical_cert.pdf ✓
 - Sick balance after this: 6 days
- [Confirm ☐] [Edit ☐] [Cancel ☒

User: (taps Confirm)

Bot: ☒ Submitted. Request #AB-10427 sent to your manager (Asha) for approval.

Clarification path

User: I'll be out next week.

Bot: Happy to help – which days next week, and what type of leave (casual, earned, sick)?

User: Mon and Tue, earned leave.

Bot: [shows confirmation card for 15–16 Jun, Earned Leave]

Policy catch (deterministic, not the LLM)

User: 3 days sick leave from Monday.

Bot: Sick leave over 2 days needs a medical certificate. Please attach one to proceed.

8. Integration with the existing system

Two options, pick by what your HRMS exposes:

1. **API integration (preferred).** PeopleSoft *Component Interface* / REST service to create the absence record and read balances. Clean, auditable, no UI fragility.
2. **RPA fallback.** If no API is available, a headless automation drives the existing *Apply / Schedule Leave* page (the one in presentUI.JPG) using the service account: fills Start Date, selects Absence Name, writes Comments, uploads the attachment, submits. Faster to pilot, more brittle to UI changes.

Either way the **approval workflow stays unchanged** — the request enters the same queue your managers already use, so no change management on their side for the MVP.

9. Security, privacy & compliance

- **Identity:** employee resolved from Azure AD SSO — users can only file leave for themselves.
 - **Least privilege:** the HRMS service account can create absences and read the requester's own balance, nothing more.
 - **Confirmation gate:** no leave is ever submitted without explicit user confirmation.
 - **Data handling:** messages may contain health info (sick leave). Keep LLM calls within an approved region/tenant, log minimally, and set retention limits. Avoid sending attachments to the LLM — metadata only.
 - **Audit trail:** store original message, parsed JSON, validations, and final submission per request.
 - **Failure mode:** if the LLM or HRMS is unavailable, the bot falls back to a link to the existing form — the manual path is never removed during pilot.
-

10. Suggested tech stack

Concern	Choice
Channel	Microsoft Teams app + Azure Bot Service (Bot Framework SDK)
Cards / UX	Adaptive Cards (confirm/edit buttons)
Orchestrator	Python (FastAPI) — clean fit with the AI SDK
LLM	Claude (c laude - opus - 4 - 8 for quality, c laude - sonnet - 4 - 6 for cost) via Anthropic SDK, tool-use for structured output
Identity	Azure AD / Entra ID SSO
State	Redis (sessions) + PostgreSQL (audit)
HRMS link	PeopleSoft CI/REST, or RPA (Playwright) fallback
Hosting	Azure App Service / Container Apps (same tenant as Teams)

11. Phased roadmap

Phase	Scope	Outcome
0 — Spike (1–2 wks)	LLM extraction only: paste a sentence, get the structured JSON. No Teams, no HRMS.	Prove parsing quality on real phrasings
1 — MVP	Teams bot + LLM + confirmation card + RPA submit ; sick/casual/earned only	One leave type family, end-to-end in Teams
2 — Robust	API integration, balance/overlap/holiday checks, half-days, attachments, doc-required rules	Production-grade validation
3 — Scale	All leave codes, cancellation/modification, balance & history queries in chat, analytics	Full self-service assistant
4 — Optional	Add Gmail adapter on the same orchestrator; manager-side AI summaries	Multi-channel

12. Risks & mitigations

Risk	Mitigation
Wrong date/type extracted	Mandatory confirmation card; show resolved dates explicitly; closed enums
Hallucinated leave code	Backend validates against live HRMS code list; unknown → clarify
Ambiguous messages	Confidence + clarifying questions before any submission
HRMS has no API	RPA fallback on the existing UI for the pilot
Sensitive health data	Region-locked LLM, minimal logging, retention limits, no attachments to LLM
LLM/cost/latency	Use Sonnet for routine parsing; cache the code list; keep prompts small

13. Next step

Recommended starting point is **Phase 0**: a tiny script that takes a sentence + the allowed leave-code list and returns the structured JSON in §5. It de-risks the whole idea in days and produces test cases for the Teams bot. Say the word and I'll build that spike (runnable, with the Claude tool schema wired up).